

CNN y GRU con Keras: diseño, evaluación y análisis de sesgo y varianza

Trabajo Práctico Integrador: Redes Neuronales | Inteligencia Artificial, Licenciatura en Sistemas de Información, FCyT-UADER

Integrantes: Facundo Tomás Hernández, Facundo Thomas Ramirez | Docentes: BiolIng. Alejandro Hadad, AS. Joaquín López y Mg. Santiago Díaz | Entrega: 30 de septiembre de 2026

1. Objetivo y protocolo

Se implementaron dos tareas independientes: clasificación espacial de dígitos con una red convolucional (CNN) y pronóstico de un paso sobre una serie multivariada con una unidad recurrente con compuertas (GRU). Se fijaron semillas y todas las decisiones (presupuesto de épocas, stride, tipo de celda) se tomaron solo con datos de desarrollo o validación. Cada conjunto de prueba se evaluó una única vez, después de fijar los pesos. Se exportaron los modelos completos y sus pesos.

2. Módulo CNN: datos y partición

Se utilizaron 1797 imágenes de dígitos manuscritos de scikit-learn (diez clases). Cada imagen de 8 x 8 se interpoló a 32 x 32 y se asignó una sola vez a uno de doce lotes de adquisición simulados, equilibrados por clase, con brillo, contraste y desenfoque propios, más ruido reproducible por muestra. Los lotes 0 a 7 forman desarrollo (1207 imágenes) y los lotes 8 a 11 se reservan para prueba (590 imágenes). Dentro de desarrollo, GroupKFold de cuatro particiones deja dos lotes completos para validación en cada iteración: ninguna imagen ni lote cruza las fronteras. Como cada lote contiene las diez clases en proporciones similares, sus condiciones de captura no están asociadas a la etiqueta; la validación por grupos mide entonces la robustez frente a condiciones de adquisición no vistas. El conjunto no incluye identificadores de autor, por lo que no demuestra generalización a escritores nuevos.

3. Arquitectura CNN y justificación

Cada activación de una convolución discreta 2D es $y[i, j] = \sum_{m,n,c} w[m, n, c] \cdot x[S \cdot i + m, S \cdot j + n, c] + b$: depende solo de una vecindad de $K \times K$ píxeles (localidad espacial) y usa los mismos pesos en todas las posiciones. Por eso una capa tiene $(K \cdot K \cdot C_{\text{entrada}} + 1) \cdot C_{\text{salida}}$ parámetros: la segunda convolución usa $(3 \cdot 3 \cdot 32 + 1) \cdot 64 = 18.496$, mientras que una capa densa entre los mismos mapas ($16 \cdot 16 \cdot 32$ entradas y $16 \cdot 16 \cdot 64$ salidas) requeriría más de 134 millones. El tamaño de salida es $O = \text{floor}((H + 2P - K) / S) + 1$. Con $K = 3$ y padding 'same' ($P = 1$), $S = 1$ conserva el tamaño ($O = H$) y $S = 2$ lo reduce a la mitad; el max pooling ($K = 2$, $S = 2$, $P = 0$) también divide por dos. La tabla muestra la secuencia de tamaños 32, 16, 8 y 4, y el campo receptivo, que crece según $r_i = r_{i-1} + (K - 1) \cdot j_{i-1}$, con $j_i = j_{i-1} \cdot S$: cada unidad de la última convolución observa 18 x 18 píxeles, suficiente para cubrir trazos completos del dígito.

Capa	Salida	Parámetros	Campo receptivo
Conv2D 32 filtros 3x3, S=1, same	32 x 32 x 32	320	3x3
MaxPooling 2x2, S=2	16 x 16 x 32	0	4x4
Conv2D 64 filtros 3x3, S=1, same	16 x 16 x 64	18.496	8x8
MaxPooling 2x2, S=2	8 x 8 x 64	0	10x10
Conv2D 64 filtros 3x3, S=2, same	4 x 4 x 64	36.928	18x18
Global average pooling	64	0	imagen completa
Densa 64, ReLU	64	4.160	-
Dropout 0,25	64	0	-
Densa 10, softmax	10	650	-

Total: 60.554 parámetros entrenables.

La convolución con $S = 1$ es equivariante a traslaciones: desplazar la entrada desplaza el mapa de salida. El pooling y el stride 2 conservan esa propiedad solo para desplazamientos múltiplos del paso y agregan invariancia local a corrimientos pequeños; de ahí que la equivarianza sea parcial. La última reducción usa una convolución con stride 2 en lugar de un tercer pooling: el submuestreo se aprende y la capa evalúa 16 posiciones en lugar de 64, con un cuarto de las multiplicaciones. Con el mismo protocolo, la variante con $S = 1$ obtuvo $0,9752 \pm 0,0118$ de exactitud de validación y la elegida $0,9751 \pm 0,0050$: igual media y menor dispersión entre lotes.

ReLU tiene derivada 1 para entradas positivas, por lo que no satura y no atenúa el gradiente al atravesar capas. La salida softmax con entropía cruzada da un gradiente respecto de cada logit igual a $p_k - y_k$, acotado entre -1 y 1. Global average pooling reemplaza el aplanado: la capa densa recibe 64 valores en lugar de 1.024. Dropout 0,25 reduce la coadaptación de unidades. Se usa Adam con tasa 0,001, lotes de 32 imágenes (unas 28 actualizaciones por época), recorte de la norma del gradiente a 1,0 y early stopping sobre la pérdida de validación con paciencia 12 y un máximo de 150 épocas, restituyendo los mejores pesos.

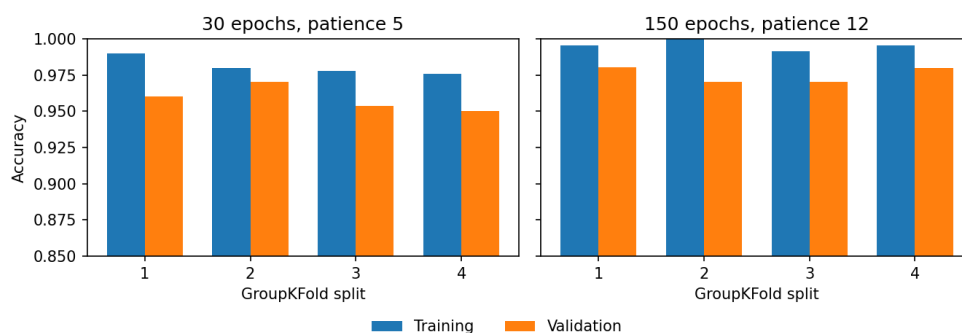
4. CNN: análisis de sesgo y varianza

Se compararon tres configuraciones con las mismas particiones GroupKFold y una referencia lineal (regresión logística sobre los píxeles). La brecha es la diferencia media entre exactitud de entrenamiento y de validación.

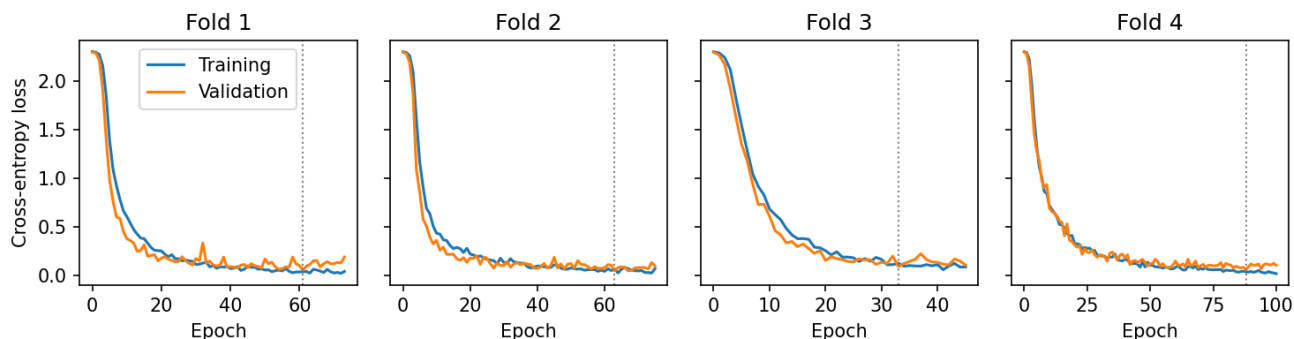
Configuración	Mejores épocas	Entrenamiento	Validación	Brecha
Presupuesto corto: 30 épocas, paciencia 5	30, 28, 30, 30	0,9809	0,9586 ± 0,0077	0,0224
Elegida: 150 épocas, paciencia 12, S = 2	62, 64, 34, 89	0,9956	0,9751 ± 0,0050	0,0205
Variante: 150 épocas, paciencia 12, S = 1	48, 58, 51, 85	0,9868	0,9752 ± 0,0118	0,0116
Referencia: regresión logística	-	-	0,9602	-

Sesgo. Con el presupuesto corto, los cuatro folds alcanzan el tope de 30 épocas con la mejor época en 28-30: la pérdida de validación seguía bajando cuando se cortó. La exactitud de validación (0,9586) queda por debajo de la referencia lineal (0,9602), aunque la CNN tiene mucha más capacidad. Ese resultado indica sesgo por entrenamiento incompleto, no por falta de capacidad. Con el presupuesto elegido, las mejores épocas van de 34 a 89, la exactitud de entrenamiento llega a 0,9956 y la de validación sube a 0,9751, por encima de la referencia lineal.

Varianza. La brecha media de la configuración elegida es 0,0205: el modelo ajusta casi por completo el entrenamiento y pierde 2,1 puntos porcentuales en lotes no vistos. En las curvas, la pérdida de entrenamiento sigue bajando mientras la de validación alcanza un mínimo (línea punteada) y luego oscila o sube: es el inicio del sobreajuste, que early stopping corta restituyendo los pesos del mínimo. La desviación entre folds (0,0050) indica baja sensibilidad a qué lotes se usan para validar.



Exactitud de entrenamiento y validación por fold, con lotes excluyentes, para el presupuesto corto y el elegido.



Pérdida de entropía cruzada por época en cada fold de la configuración elegida. La línea punteada marca la época restituida por early stopping.

Modelo final y prueba. Se reentrenó con todo el desarrollo durante 63 épocas (mediana de las mejores épocas) y se evaluó una vez sobre los lotes 8 a 11: exactitud 0,9763 y F1 macro 0,9764, con 14 errores sobre 590 imágenes. La exactitud de prueba coincide con la media de validación cruzada, por lo que los lotes reservados no revelan degradación. Las confusiones más frecuentes son 5 clasificado como 9 (2 casos) y 3 clasificado como 9 (2 casos); la matriz completa está en `cnn_metrics.json`.

5. Módulo GRU: serie y corte temporal

Se generó una serie sintética de 1600 instantes con un componente autorregresivo, seno y coseno estacionales (período 48), un control correlacionado y ruido gaussiano de desviación 0,11. El objetivo en t depende de su valor en $t-1$, de la estación en t y del control en $t-1$. Cada entrada contiene los 36 pasos anteriores de las cuatro variables y el objetivo es el valor siguiente. Los tramos son entrenamiento [0, 960), validación [960, 1280) y prueba [1280, 1600). Todas las ventanas se construyen dentro de su propio tramo: se descartan los primeros 36 objetivos de cada frontera para que ninguna ventana de validación o prueba use historia de un tramo anterior. La media y la desviación de las entradas y del objetivo se ajustan solo con entrenamiento y se guardan en `gru_scalers.npz`.

6. Arquitectura GRU y estabilidad del gradiente

En BPTT el gradiente hacia un estado pasado es $dL/dh_k = dL/dh_T \cdot J_T \cdot J_{T-1} \cdots J_{k+1}$, con $J_t = dh_t/dh_{t-1}$. En una RNN simple, $h_t = \tanh(W x_t + U h_{t-1} + b)$ y $J_t = \text{diag}(1 - h_t^2) \cdot U$. Como la derivada de $\tanh$ no supera 1, la norma del producto está acotada por $\|U\|^{T-k}$: decae geométricamente si $\|U\| < 1$ (desaparición) y puede crecer sin límite si es mayor (explosión).

La GRU calcula una compuerta de actualización $z_t = \text{sigmoid}(W_z x_t + U_z h_{t-1})$, una de reinicio r_t y un candidato $g_t = \tanh(W x_t + U(r_t \cdot h_{t-1}))$, y combina $h_t = z_t \cdot h_{t-1} + (1 - z_t) \cdot g_t$ (productos elemento a elemento). Entonces $J_t = \text{diag}(z_t) + (\text{términos que pasan por } g_t \text{ y } z_t)$: el primer término es una vía aditiva que no se multiplica por U ni por la derivada de $\tanh$. Cuando la red aprende z_t cercano a 1, ese factor se aproxima a la identidad y el gradiente atraviesa muchos pasos sin atenuarse. Esto mitiga la desaparición, aunque no la elimina. Contra la explosión, el optimizador recorta la norma del gradiente de cada tensor de pesos a 1,0 (g se reemplaza por $g \cdot \min(1, 1/\|g\|)$). La $\tanh$ mantiene el estado acotado en $(-1, 1)$ y las compuertas sigmoid quedan en $(0, 1)$.

Para evaluar la elección se entrenaron las tres celdas con 32 unidades, la misma cabeza (densa de 16 unidades ReLU y salida lineal), pérdida MSE, Adam con tasa 0,001, lotes de 32, orden cronológico conservado (`shuffle=False`) y early stopping con paciencia 7 sobre un máximo de 55 épocas. La selección se hizo por MAE de validación; la columna de prueba se informa después.

Celda	Parámetros	Mejor época	MAE validación	MAE prueba	RMSE prueba
GRU (elegida)	4.193	45	0,0891	0,0960	0,1201
LSTM	5.281	54	0,0949	0,1000	0,1244
RNN simple	1.729	24	0,0896	0,0981	0,1223

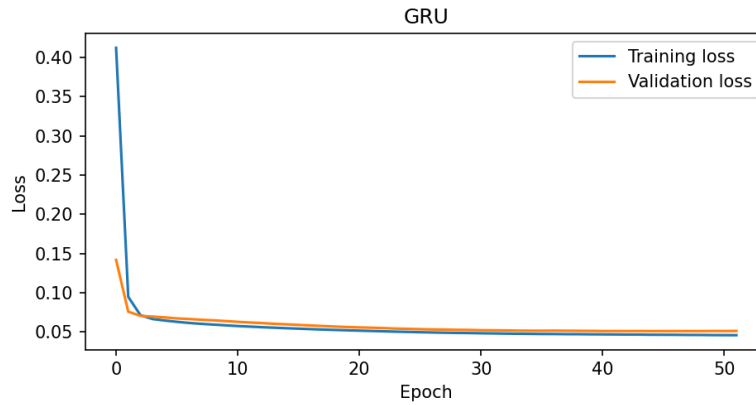
La GRU obtiene el menor error de validación (0,0891) con 21% menos parámetros que la LSTM. La RNN simple queda muy cerca porque la dinámica de esta serie es de corto alcance (el objetivo depende del paso anterior): aquí la propagación del gradiente a lo largo de muchos pasos no es el factor limitante. La ventaja de las compuertas aparece cuando las dependencias abarcan muchos pasos; la GRU ofrece esa protección con menos parámetros que la LSTM. La ventana de 36 pasos cubre tres cuartos del ciclo estacional. La salida lineal no impone un rango artificial a la regresión y MSE penaliza más los errores grandes, en línea con el RMSE informado.

7. Resultados observados: GRU

Como la serie es sintética, se conoce la parte predecible del proceso: aplicar la ecuación verdadera deja solo el ruido, cuyo error fija un piso que ningún modelo puede superar en promedio. La persistencia (repetir el último valor observado) sirve como referencia ingenua.

Modelo	MAE prueba	RMSE prueba
GRU	0,0960	0,1201
Persistencia	0,1139	0,1428
Piso de ruido (ecuación verdadera)	0,0917	0,1150

Sesgo. La GRU reduce el MAE de la persistencia en 15,7% y queda solo 4,7% por encima del piso de ruido: casi todo el error remanente es irreducible, por lo que el sesgo es bajo. **Varianza.** El MAE es 0,0841 en entrenamiento, 0,0891 en validación y 0,0960 en prueba; las brechas son pequeñas frente al error medio y en la curva ambas pérdidas convergen sin divergir. **Estabilidad predictiva.** El MAE por cuartos cronológicos de la prueba es 0,0987, 0,1002, 0,0947, 0,0902: no crece a medida que la prueba se aleja del entrenamiento.



Pérdida MSE por época en entrenamiento y validación cronológica de la GRU (mejor época: 45).

8. Limitaciones

La CNN se evalúa sobre lotes simulados, no sobre personas o dispositivos nuevos. La serie de la GRU es un proceso sintético estacionario de dependencia corta, útil para verificar el protocolo pero no extrapolable a un dominio aplicado. No se reclaman conclusiones para otras poblaciones. Los resultados se reproducen con el código; distintas CPU o versiones de bibliotecas pueden introducir diferencias numéricas pequeñas.

9. Archivos, reproducción y referencias

`pip install -r requirements.txt`; `python train.py`; `python predict.py`; `python -m unittest discover -s tests -v`; `python make_report.py`. `cnn.keras` y `gru.keras` contienen arquitectura y pesos; `*.weights.h5` son los pesos; `gru_scalers.npz` se necesita para interpretar la salida de la GRU; `cnn_metrics.json` y `gru_metrics.json` guardan todos los resultados. Referencias: documentación de Keras (keras.io/api) y scikit-learn (scikit-learn.org/stable); Goodfellow, Bengio y Courville, Deep Learning, MIT Press, 2016, capítulos 9 y 10; Cho et al., Learning Phrase Representations using RNN Encoder-Decoder, EMNLP 2014.